1) PRIMARY KEY add karna

SQL

```sql
ALTER TABLE student
ADD PRIMARY KEY (id);
```

---

2) COMPOSITE PRIMARY KEY add karna

Agar 2 columns milke primary key banani ho:

SQL

```sql
ALTER TABLE enrollment
ADD PRIMARY KEY (student_id, course_id);
```

1. Numeric Data Types

Data Type	Size	Range / Use
`TINYINT`	1 byte	-128 to 127
`SMALLINT`	2 bytes	-32,768 to 32,767
`MEDIUMINT`	3 bytes	-8M to 8M
`INT` / `INTEGER`	4 bytes	-2B to 2B
`BIGINT`	8 bytes	बहुत बड़े integers
`FLOAT`	4 bytes	Approximate decimal
`DOUBLE`	8 bytes	More precision
`DECIMAL(M,D)`	Variable	Exact decimal values

2. String Data Types

Data Type	Size	Use
`CHAR(n)`	Fixed	Fixed-length text
`VARCHAR(n)`	Variable	Variable-length text
`TINYTEXT`	Up to 255 bytes	Short text
`TEXT`	Up to 65,535 bytes (≈64 KB)	Paragraphs
`MEDIUMTEXT`	Up to 16 MB	Large text
`LONGTEXT`	Up to 4 GB	Very large text

3. Binary Data Types

Data Type	Use
`BINARY(n)`	Fixed binary data
`VARBINARY(n)`	Variable binary data
`TINYBLOB`	Up to 255 bytes
`BLOB`	Up to 64 KB
`MEDIUMBLOB`	Up to 16 MB
`LONGBLOB`	Up to 4 GB

BLOB का उपयोग images, PDFs, audio जैसी binary files के लिए किया जाता है।

4. Date & Time Data Types

Data Type	Size	Example
`DATE`	3 bytes	`2026-07-13`
`TIME`	3 bytes	`14:30:45`
`YEAR`	1 byte	`2026`
`DATETIME`	5 bytes	`2026-07-13 14:30:45`
`TIMESTAMP`	4 bytes	Auto timestamp

7. ENUM Data Type

केवल predefined values store करता है।

```SQL
gender ENUM('Male','Female','Other')
```

---

8. SET Data Type

एक से अधिक predefined values store कर सकता है।

```SQL
skills SET('C++','Java','Python','SQL')
```

9. Spatial Data Types (GIS)

Data Type	Use
`POINT`	Single location
`LINESTRING`	Road/Path
`POLYGON`	Area
`GEOMETRY`	Generic geometry

1-byte character set (Interview के लिए आसान table)

Data Type	Maximum Characters	Maximum Bytes
`CHAR(255)`	255	255 bytes
`VARCHAR`	लगभग 65,535 characters*	लगभग 65,535 bytes*
`TINYTEXT`	255	255 bytes
`TEXT`	65,535	65,535 bytes (≈64 KB)
`MEDIUMTEXT`	16,777,215	16,777,215 bytes (≈16 MB)
`LONGTEXT`	4,294,967,295	4,294,967,295 bytes (≈4 GB)

Data Type	Max Size
`TINYTEXT`	**255 B**
`TEXT`	**64 KB**
`MEDIUMTEXT`	**16 MB**
`LONGTEXT`	**4 GB**

3) UNIQUE KEY add karna

SQL

```sql
ALTER TABLE student
ADD UNIQUE (email);
```

Agar constraint name bhi dena ho:

SQL

```sql
ALTER TABLE student
ADD CONSTRAINT uq_email UNIQUE (email);
```

4) FOREIGN KEY add karna

Example: `student.dept_id` ka relation `department.id` se hai

SQL

```sql
ALTER TABLE student
ADD CONSTRAINT fk_dept
FOREIGN KEY (dept_id)
REFERENCES department(id);
```

---

5) COMPOSITE UNIQUE KEY add karna

SQL

```sql
ALTER TABLE employee
ADD CONSTRAINT uq_emp UNIQUE (name, phone);
```

1. Add Column

SQL

```sql
ALTER TABLE student
ADD email VARCHAR(100);
```

---

2. Drop Column

SQL

```sql
ALTER TABLE student
DROP COLUMN email;
```

3. Rename Table

SQL

```sql
RENAME TABLE student TO students;

-- OR
ALTER TABLE students
RENAME TO student;
```

---

4. Rename Column (MySQL 8.0+)

SQL

```sql
ALTER TABLE student
RENAME COLUMN phone TO mobile;
```

5. Change Column (Rename + Datatype)

SQL

```sql
ALTER TABLE student
CHANGE COLUMN mobile phone VARCHAR(15);
```

Old Name → `mobile`

New Name → `phone`

---

6. Modify Column (Datatype Only)

SQL

```sql
ALTER TABLE student
MODIFY COLUMN phone VARCHAR(20);
```

7. Change Column Position

First Position

SQL

```sql
ALTER TABLE student
MODIFY COLUMN phone VARCHAR(15) FIRST;
```

After Another Column

SQL

```sql
ALTER TABLE student
MODIFY COLUMN phone VARCHAR(15) AFTER name;
```

Solution 1 (Recommended)

SQL

```sql
TRUNCATE TABLE student;
TRUNCATE TABLE department;
```

Solution 2 (Disable FK Checks)

SQL

```sql
SET FOREIGN_KEY_CHECKS = 0;

TRUNCATE TABLE department;

SET FOREIGN_KEY_CHECKS = 1;
```

Correct Order

SQL

```sql
DROP TABLE student;

DROP TABLE department;
```

OR

SQL

```sql
SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE department;

SET FOREIGN_KEY_CHECKS = 1;
```

Operation	Query
Add Column	`ALTER TABLE student ADD email VARCHAR(100);`
Drop Column	`ALTER TABLE student DROP COLUMN email;`
Rename Table	`RENAME TABLE student TO students;`
Rename Column	`ALTER TABLE student RENAME COLUMN phone TO mobile;`
Change Column	`ALTER TABLE student CHANGE mobile phone VARCHAR(15);`
Modify Column	`ALTER TABLE student MODIFY phone VARCHAR(20);`
First Position	`ALTER TABLE student MODIFY phone VARCHAR(15) FIRST;`
After Column	`ALTER TABLE student MODIFY phone VARCHAR(15) AFTER name;`

After Column	`ALTER TABLE student MODIFY phone VARCHAR(15) AFTER name;`
Truncate Child	`TRUNCATE TABLE student;`
Truncate Parent	`TRUNCATE TABLE department;` *(fails if referenced by FK)*
Delete Child	`DELETE FROM student;`
Delete Parent	`DELETE FROM department;` *(works with* `ON DELETE CASCADE`*, otherwise fails)*
Drop Child	`DROP TABLE student;`
Drop Parent	`DROP TABLE department;` *(fails if referenced by FK unless child removed or FK checks disabled)*

4) Undo / Rollback

Ye depend karta hai **kis type ka command** chalaya:

A) `INSERT / UPDATE / DELETE` ka undo

Agar transaction use ki hai aur `COMMIT` nahi kiya:

```sql
START TRANSACTION;

UPDATE student
SET name = 'Rahul'
WHERE id = 1;

ROLLBACK;
```

B) `CREATE / DROP / ALTER / RENAME` ka undo?

Normally **DDL commands** (`CREATE` , `DROP` , `ALTER` , `RENAME`) MySQL me **auto-commit** kar dete hain.
Matlab:

- `RENAME TABLE` → rollback nahi hota
- `ALTER TABLE` → rollback nahi hota
- `DROP TABLE` → rollback nahi hota
- `DROP DATABASE` → rollback nahi hota

Savepoint transaction ke andar ek **checkpoint** hota hai.
Iska use tab hota hai jab tum poori transaction rollback nahi karna chahte, **sirf ek specific point tak undo** karna chahte ho.

Simple meaning

Maan lo tumne transaction me 3 queries chalayi:

1. pehla update sahi hai
2. dusra update sahi hai
3. teesra galat ho gaya

Ab agar tum **ROLLBACK** karoge to **sab kuch undo** ho jayega.
Lekin agar tumne बीच में **SAVEPOINT** banaya tha, to tum **sirf us savepoint tak rollback** kar sakte ho.

1) Transaction start

```sql
START TRANSACTION;
```

2) Savepoint banana

```sql
SAVEPOINT sp1;
```

3) Savepoint tak rollback

```sql
ROLLBACK TO sp1;
```

SQL

```sql
START TRANSACTION;

UPDATE student
SET age = 21
WHERE id = 1;

SAVEPOINT sp1;

UPDATE student
SET age = 22
WHERE id = 2;

UPDATE student
SET age = 23
WHERE id = 3;

ROLLBACK TO sp1;

COMMIT;
```

Savepoint vs Rollback

`ROLLBACK`

poori transaction undo

```sql
START TRANSACTION;

UPDATE student SET age=20 WHERE id=1;
UPDATE student SET age=25 WHERE id=2;

ROLLBACK;
```

👉 dono updates undo

`ROLLBACK TO savepoint`

sirf savepoint ke baad wale changes undo

SQL

```sql
START TRANSACTION;

UPDATE student SET age=20 WHERE id=1;

SAVEPOINT sp1;

UPDATE student SET age=25 WHERE id=2;

ROLLBACK TO sp1;
COMMIT;
```

👉 id=1 update rahega

👉 id=2 undo ho jayega

Insert uppercase

SQL

```sql
INSERT INTO student(id, name, city)
VALUES (1, UPPER('tarun'), UPPER('indore'));
```

Insert lowercase

SQL

```sql
INSERT INTO student(id, name, city)
VALUES (2, LOWER('RAHUL'), LOWER('BHOPAL'));
```

Sabko uppercase

SQL

```sql
UPDATE student
SET name = UPPER(name),
    city = UPPER(city);
```

Sabko lowercase

SQL

```sql
UPDATE student
SET name = LOWER(name),
    city = LOWER(city);
```

Function	Purpose	Example
`UPPER()`	Uppercase	`SELECT UPPER('tarun');` → `TARUN`
`LOWER()`	Lowercase	`SELECT LOWER('TARUN');` → `tarun`
`LENGTH()`	String ki length	`SELECT LENGTH('Tarun');` → `5`
`CONCAT()`	Strings jodna	`SELECT CONCAT('Tarun',' Patidar');` → `Tarun Patidar`
`SUBSTRING()`	String ka part nikalna	`SELECT SUBSTRING('Tarun',2,3);` → `aru`
`LEFT()`	Left se characters	`SELECT LEFT('Tarun',3);` → `Tar`
`RIGHT()`	Right se characters	`SELECT RIGHT('Tarun',2);` → `un`
`TRIM()`	Dono side ke spaces remove	`SELECT TRIM(' Tarun ');` → `Tarun`
`LTRIM()`	Left spaces remove	`SELECT LTRIM(' Tarun');`
`RTRIM()`	Right spaces remove	`SELECT RTRIM('Tarun ');`
`REPLACE()`	String replace karna	`SELECT REPLACE('Tarun','Tar','Kar');` → `Karun`
`REVERSE()`	String reverse	`SELECT REVERSE('Tarun');` → `nuraT`
`INSTR()`	Character ki position	`SELECT INSTR('Tarun','r');` → `3`
`LOCATE()`	Character/String ki position	`SELECT LOCATE('ru','Tarun');` → `3`

Sql full

```sql
CREATE TABLE student (
    id INT PRIMARY KEY,

    phone VARCHAR(10),
    CHECK (phone REGEXP '^[6-9][0-9]{9}$'),

    aadhaar CHAR(12),
    CHECK (aadhaar REGEXP '^[0-9]{12}$'),

    dob VARCHAR(10),
    CHECK (dob REGEXP '^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/[0-9]{4}$'),

    password VARCHAR(16),
    CHECK (
        CHAR_LENGTH(password) = 16
        AND password REGEXP '[A-Z]'
        AND password REGEXP '[a-z]'
        AND password REGEXP '[0-9]'
        AND password REGEXP '[@#$]'
    )
);
```

```sql
CHECK (
CHAR_LENGTH(password)=16
AND password REGEXP '[A-Z]'
AND password REGEXP '[a-z]'
AND password REGEXP '[0-9]'
AND password REGEXP '[@#$]'
)
```

1. Delete Row

SQL

```sql
DELETE FROM student
WHERE id = 1;
```

Undo (Transaction ke andar)

SQL

```sql
START TRANSACTION;

DELETE FROM student
WHERE id = 1;

ROLLBACK;
```

Delete one row

SQL

```sql
DELETE FROM student WHERE id = 1;
```

Delete all rows

SQL

```sql
DELETE FROM student;
```

Delete column

SQL

```sql
ALTER TABLE student DROP COLUMN phone;
```

```sql
-- Sirf ek record delete
DELETE FROM student WHERE id = 1;

-- A, B, C delete
DELETE FROM student WHERE name IN ('A','B','C');

-- B ko chhodkar sab delete
DELETE FROM student WHERE name <> 'B';

-- A, B, C ko chhodkar sab delete
DELETE FROM student WHERE name NOT IN ('A','B','C');

-- Age 18 se kam delete
DELETE FROM student WHERE age < 18;

-- NULL values delete
DELETE FROM student WHERE email IS NULL;
```

1. Equal (=)

SELECT

SQL

```sql
SELECT * FROM student
WHERE age = 20;
```

UPDATE

SQL

```sql
UPDATE student
SET city = 'Indore'
WHERE age = 20;
```

DELETE

SQL

```sql
DELETE FROM student
WHERE age = 20;
```

---

2. Not Equal (!=)

SELECT

SQL

```sql
SELECT * FROM student
WHERE age != 20;
```

UPDATE

SQL

```sql
UPDATE student
SET city = 'Bhopal'
WHERE age != 20;
```

DELETE

SQL

```sql
DELETE FROM student
WHERE age != 20;
```

3. Not Equal (<>)

SELECT

SQL

```sql
SELECT * FROM student
WHERE age <> 20;
```

UPDATE

SQL

```sql
UPDATE student
SET city = 'Ujjain'
WHERE age <> 20;
```

UPDATE

SQL

```sql
UPDATE student
SET city = 'Ujjain'
WHERE age <> 20;
```

DELETE

SQL

```sql
DELETE FROM student
WHERE age <> 20;
```

4. Greater Than (>)

SELECT

SQL

```sql
SELECT * FROM student
WHERE age > 20;
```

UPDATE

SQL

```sql
UPDATE student
SET city = 'Delhi'
WHERE age > 20;
```

```sql
SELECT *
FROM student
WHERE city = 'Indore'
AND marks > 80;
```

👉 Sirf wahi students aayenge jo **Indore** ke hain **aur** jinke marks **80 se zyada** hain.

UPDATE

```sql
UPDATE student
SET marks = marks + 5
WHERE city = 'Indore'
AND course_year = 1;
```

👉 Sirf **Indore ke 1st year** students ke marks 5 se inc e honge.

```sql
DELETE FROM student
WHERE city = 'Dhar'
AND marks < 75;
```

👉 Sirf Dhar ke students jinke marks 75 se kam hain delete honge.

---

2. Bitwise AND (`&`)

Meaning: Numbers ke **binary bits** compare karta hai.

Example:

```
5 = 101
3 = 011
---------
    001 = 1
```

Odd numbers ka last bit `1` hota hai.

SQL

```sql
SELECT *
FROM student
WHERE (marks & 1) = 1;
```

👉 Jinke marks **odd** hain (69, 91, 83...) wo students aayenge.

Even marks:

SQL

```sql
SELECT *
FROM student
WHERE (marks & 1) = 0;
```

👉 Jinke marks **even** hain (78, 82, 84...) wo students ↓ nge.

UPDATE

SQL

```sql
UPDATE student
SET marks = marks + 2
WHERE (marks & 1) = 1;
```

👉 Sirf odd marks wale students ke marks 2 se badhenge.

DELETE

SQL

```sql
DELETE FROM student
WHERE (marks & 1) = 0;
```

👉 Sirf even marks wale students delete ho jayenge.

Ek aur Bitwise AND Example

IDs me bitwise AND:

SQL

```sql
SELECT id, name
FROM student
WHERE (id & 1) = 1;
```

👉 Odd IDs:

```
1
3
5
7
...
65
```

```sql
SELECT *
FROM student
WHERE city='Indore'
OR marks>90;
```

Output:

- Indore ke saare students
- Ya jinke marks 90 se jyada hain

UPDATE

```sql
UPDATE student
SET marks=100
WHERE city='Indore'
OR marks>90;
```

SELECT

SQL

```sql
SELECT *
FROM student
WHERE NOT city='Indore';
```

Output:

Indore ko chhodkar baaki sab.

UPDATE

SQL

```sql
UPDATE student
SET city='Delhi'
WHERE NOT city='Indore';
```

Indore ke alawa sabki city Delhi ho jayegi.

DELETE

```sql
DELETE FROM student
WHERE NOT city='Indore';
```

Indore ko chhodkar sab delete.

---

3. NOR

SQL me **NOR operator nahi hota.**

NOR ka matlab hota hai

```
NOT(A OR B)
```

---

1. IN Operator

Definition

IN operator multiple values me se match karta hai.

Iska matlab:

```sql
WHERE city='Indore'
OR city='Bhopal'
OR city='Ujjain'
```

ko short me likhte hain:

```sql
WHERE city IN ('Indore','Bhopal','Ujjain')
```

SELECT

SQL

```sql
SELECT *
FROM student
WHERE city IN ('Indore','Bhopal','Ujjain');
```

👉 Indore, Bhopal aur Ujjain ke students.

UPDATE

SQL

```sql
UPDATE student
SET marks = marks + 5
WHERE city IN ('Indore','Bhopal');
```

👉 Indore aur Bhopal ke students ke marks 5 se bad[illegible]yenge.

2. NOT IN Operator

Definition

List me jo values hain unhe chhodkar baaki sab select karega.

SELECT

```SQL
SELECT *
FROM student
WHERE city NOT IN ('Indore','Bhopal');
```

👉 Indore aur Bhopal ko chhodkar baaki sab students.

UPDATE

SQL

```sql
UPDATE student
SET marks = 50
WHERE city NOT IN ('Indore','Bhopal');
```

👉 Indore aur Bhopal ke alawa sabke marks 50 ho jayenge.

DELETE

SQL

```sql
DELETE FROM student
WHERE city NOT IN ('Indore','Bhopal');
```

👉 Sirf Indore aur Bhopal ke students bachenge.

3. BETWEEN Operator

Definition

Range ke andar ki values select karta hai.

Equivalent to:

```sql
age >=18
AND age<=20
```

SELECT

```sql
SELECT *
FROM student
WHERE age BETWEEN 18 AND 20;
```

UPDATE

SQL

```sql
UPDATE student
SET marks = marks + 10
WHERE age BETWEEN 18 AND 20;
```

👉 18–20 age ke students ke marks 10 se badh jayenge.

DELETE

SQL

```sql
DELETE FROM student
WHERE age BETWEEN 18 AND 20;
```

👉 18–20 age ke students delete ho jayenge.

BETWEEN with Marks

SELECT

SQL

```sql
SELECT *
FROM student
WHERE marks BETWEEN 80 AND 90;
```

👉 80 se 90 (inclusive) marks wale students.

UPDATE

SQL

```sql
UPDATE student
SET city='Indore'
WHERE marks BETWEEN 80 AND 90;
```

DELETE

SQL

```sql
DELETE FROM student
WHERE marks BETWEEN 80 AND 90;
```

NOT BETWEEN

SQL

```sql
SELECT *
FROM student
WHERE age NOT BETWEEN 18 AND 20;
```

👉 18–20 ke alawa sab.

1. LIKE Operator

Definition

LIKE string (text) me **pattern search** karne ke liye use hota hai.

Wildcards:

- `%` → 0 ya usse zyada characters
- `_` → Exactly 1 character

`%` (Zero or More Characters)

A se start

```sql
SELECT *
FROM student
WHERE name LIKE 'A%';
```

Match:

```
Aarav
Aman
Aditya
```

n se end

SQL

```sql
SELECT *
FROM student
WHERE name LIKE '%n';
```

Match:

```
Aman
```

"ar" contain kare

SQL

```sql
SELECT *
FROM student
WHERE name LIKE '%ar%';
```

Match:

```
Tarun
```

Pattern	Meaning
`'A%'`	Starts with A
`'%A'`	Ends with A
`'%A%'`	Contains A
`'A____'`	A + total 5 chars
`'_____'`	Exactly 5 chars
`'_a%'`	Second letter a

```sql
SELECT *
FROM student
WHERE marks > ANY
(
SELECT marks
FROM topper
);
```

ANY ka matlab

List ke kisi ek value ke saath condition true honi chahiye.

Topper list

```
60

80

95
```

ALL Operator

Same topper

```
60

80

95
```

Query

```sql
SELECT *
FROM student
WHERE marks > ALL
(
SELECT marks
FROM topper
);
```

`LIKE`	Pattern matching (`%` , `_`)
`ANY`	**Kisi ek** value ke saath condition true (`OR` jaisa)
`ALL`	**Har** value ke saath condition true (`AND` jaisa)